

## Other hardware primitives

### Compare and swap (or compare and exchange)

```
1  int CompareAndSwap(int *ptr, int expected, int new)
2  {
3      int original = *ptr;
4      if (original == expected)
5          *ptr = new;
6      return original;
7  }
```

→ Definition  
in C

Note that here, the memory location pointed to by *ptr* is updated only when the current value at the location is equal to *expected*.  
O/w do nothing. In any case, return the *original* value.

How to build a lock using CompareAndSwap?

↳ replace the *lock()* routine of TestAndset by the following code.

```
1  void lock(lock_t *lock) {
2      while (CompareAndSwap(&lock->flag, 0, 1) == 1)
3          ; // spin
4  }
```

Checks if the *flag* is 0, and if so, atomically swaps in a 1 and thus acquires the lock.

Why the above lock works?

↳ threads that try to acquire the lock while it is held will get stuck spinning until the lock is released.

\* correctness and performance of a spin lock using **CompareAndSwap** is similar to that of **TestAndSet**.

### Load-linked and store-conditional

```
1  int LoadLinked(int *ptr) {  
2      return *ptr;  
3  }  
4  
5  int StoreConditional(int *ptr, int value) {  
6      if (no update to *ptr since LL to this addr) {  
7          *ptr = value;  
8          return 1; // success!  
9      } else {  
10         return 0; // failed to update  
11     }  
12 }
```

} → fetches a value and places it in a register.

→ only succeeds if no intervening store to the address has taken place.

→ Definition in C.

How to build a lock using load-linked and store-conditional?

```

1 void lock(lock_t *lock) {
2     while (1) {
3         while (LoadLinked(&lock->flag) == 1) }
4             ; // spin until it's zero
5         if (StoreConditional(&lock->flag, 1) == 1)
6             return; // if set-to-1 was success: done
7                 // otherwise: try again
8     }
9 }
10
11 void unlock(lock_t *lock) {
12     lock->flag = 0;
13 }

```

→ thread spins waiting for flag to be set to 0.

→ once flag == 0, let thread1 tries to acquire the lock.  
 if it succeeds, it means no other thread has updated the flag  
 ⇒ no other thread has acquired the lock in b/w  
 and thus thread1 acquires the lock, and proceeds to the critical section.

when does a thread fail in this step?

let thread1 calls lock() and executes Load-linked, returns 0, as it was not held. Now, before it executes store conditional, it is interrupted by, say, thread2, enters the lock code.

thread 2 also see  $flag == 0$  and continues.

At this point, both thread 1 and thread 2 are about to attempt the store conditional.

But only one of them succeed in updating the flag.

↳ because, the second thread to attempt the store-conditional will fail (think, why?)

What is a common problem in all the above lock strategies?

↳ threads might suffer from starvation.

A hardware primitive to solve this?

Fetch-And-Add

Idea: automatically increments a value while returning the old value at a particular address.

```
1 int FetchAndAdd(int *ptr) {  
2     int old = *ptr;  
3     *ptr = old + 1;  
4     return old;  
5 }
```

→ Definition in C

we use Fetch-And-Add to build a ticket lock, as given below.



```

1  typedef struct __lock_t {
2      int ticket;
3      int turn;
4  } lock_t;
5
6  void lock_init(lock_t *lock) {
7      lock->ticket = 0;
8      lock->turn   = 0;
9  }
10
11 void lock(lock_t *lock) {
12     int myturn = FetchAndAdd(&lock->ticket);
13     while (lock->turn != myturn)
14         ; // spin
15 }
16
17 void unlock(lock_t *lock) {
18     lock->turn = lock->turn + 1;
19 }

```

→ generate a ticket value atomically. ↓

myturn

the thread that enters the critical section is the one for which  $myturn == turn$ .

increments such that the next waiting thread (if there is any) can enter the critical section.

What is the advantage here? → ensures progress for all threads. (Because, any thread who tries to get the lock, is assigned a ticket value, and will be scheduled at some point in the future).

In all the above approaches, a thread waiting for the lock, keeps spinning.

Why spinning is bad? If there are  $N$  threads contending for a lock; and an interrupt happened while the first thread holding the lock enters into the critical section, then  $N-1$  time slices may be wasted as the other waiting threads simply spin in their CPU cycle.

Question: How to avoid spinning?

What to do when a context switch occurs in a critical section, and a waiting thread starts to spin endlessly?

↳ just yield! (instead of spinning, just give up the CPU to another thread).

```
1 void init() {  
2     flag = 0;  
3 }  
4  
5 void lock() {  
6     while (TestAndSet(&flag, 1) == 1)  
7         yield(); // give up the CPU  
8 }  
9  
10 void unlock() {  
11     flag = 0;  
12 }
```

↳ an OS primitive

→ lock with Test-And-set and yield()

a system call that moves the caller from the running state to ready state.

How to avoid starvation without spinning so much?

First, to avoid starvation, with the help of OS, we maintain a **queue** to keep track of which threads are waiting for the lock.

How to avoid spinning?

Solaris used two calls

**park()** → to put a calling thread to sleep  
(ready → blocked) if it tries to acquire a held lock

**unpark(thread ID)** → to wake a particular thread as  
(blocked → ready) designated by thread ID, when the lock is free.

Idea: combine **test-and-set** with a **queue of lock waiters**  
use queue to control who gets the lock next.

Remark: Doesn't avoid spin-waiting entirely!

# lock with Queues, Test-and-set, Yield, and Wakeup.

```
1 typedef struct __lock_t {
2     int flag;
3     int guard;
4     queue_t *q;
5 } lock_t;
6
7 void lock_init(lock_t *m) {
8     m->flag = 0;
9     m->guard = 0;
10    queue_init(m->q);
11 }
12
13 void lock(lock_t *m) {
14     while (TestAndSet(&m->guard, 1) == 1)
15         ; //acquire guard lock by spinning
16     if (m->flag == 0) {
17         m->flag = 1; // lock is acquired
18         m->guard = 0;
19     } else {
20         queue_add(m->q, gettid());
21         m->guard = 0;
22         park();
23     }
24 }
25
26 void unlock(lock_t *m) {
27     while (TestAndSet(&m->guard, 1) == 1)
28         ; //acquire guard lock by spinning
29     if (queue_empty(m->q))
30         m->flag = 0; // let go of lock; no one wants it
31     else
32         unpark(queue_remove(m->q)); // hold lock
33                                     // (for next thread!)
34     m->guard = 0;
35 }
```

→ spinlock to protect the lock's internal fields → so that multiple threads don't

initially lock and guard is free and ensures the queue is empty.

modify flag or queue at the same time.

→ atomically sets guard to 1 and return the old value.

If old value = 1, someone else has the guard; spins  
o/w → got the guard  
→ safe to access flag and q.

→ ensures that once a thread holds guard = 1, no other thread can modify flag or q at the same time.

→ release guard  
lock is held by some other thread  
so add the current thread to queue; release guard; block the thread until unparked.

This implements blocking behaviour instead of just spinning forever.

Also, note that the time spent on spinning is quite limited (just a few instructions inside the lock and unlock code, instead of the user-defined critical section)

Question: On the `lock()` routine of the above code, what would happen if the release of the guard lock came after the `park()` (not before as in the code)?

Hypothetical change

```
queue_add(m->q, getpid());  
park();           // block first  
m->guard = 0;    // release guard **after blocking**
```

Consequences:

1. Thread T1 enters `lock()` first and acquires the lock.
2. Thread T2 enters `lock()`, see the lock is held
3. T2 adds itself to the queue
4. T2 calls `park()`

by our hypothetical change

⇒ T2 is blocked, but still holds `m->guard = 1`

5. T1 calls unlock()

```
while (TestAndSet(&m->guard, 1) == 1)
    ; // acquire guard
```

- T1 tries to acquire the guard but T2 is still holding it.
- T1 spins forever, because T2 is blocked in park() and cannot release the guard.

what happens then? → Deadlock occurs.

- T2 is parked and cannot release the guard
- T1 (or any other thread) cannot release the lock or unpark T2
- no thread can make progress.

So, releasing the guard before park() is essential to avoid deadlock.

to avoid race conditions (think how race condition arise o/w)  
Another remark: The flag does not get set back to 0 when another thread gets woken up. Then how does it acquire the lock?

- The lock is "paused" directly \* unlock() wakes the next thread
- \* that thread acquires the guard first, then enter the critical section.
- \* flag remains 1 the whole time.